

Spark DAG / Stage / Task 手算

先记住总规则：Spark 从逻辑到物理主要做两件事：划分 **Stage**、划分 **Task**；第一阶段 Task 数通常由 **HDFS block 或 HBase region** 个数决定，后续 Stage 的 Task 数由用户设置的 shuffle 分区数决定，未设置时通常用默认值。PPT 里也强调，Spark 执行流程是 RDD 建 DAG → 生成物理计划 → 调度 task → executor 执行。

核心口诀就三句：

看到 Action：触发一个 Job
看到 Shuffle / 宽依赖：切一个 Stage
一个 Stage 里，一个 Partition 通常对应一个 Task

题1：WordCount 的 Stage 和 Task 数

题目：HDFS 输入文件有 4 个 block，程序如下，**reduceByKey** 的分区数设为 2。问有几个 Job、几个 Stage、每个 Stage 几个 Task？

```
val lines = sc.textFile("hdfs://input")
val words = lines.flatMap(_.split(" "))
val pairs = words.map(word => (word, 1))
val counts = pairs.reduceByKey(_ + _, 2)
counts.saveAsTextFile("hdfs://output")
```

答案：

1 个 Job
2 个 Stage

Stage 1: textFile + flatMap + map + shuffle write
Task 数 = 4，因为输入 HDFS 有 4 个 block

Stage 2: shuffle read + reduceByKey + saveAsTextFile
Task 数 = 2，因为 reduceByKey 设置了 2 个 shuffle partition

详解：**flatMap** 和 **map** 都是窄依赖，不切 Stage。**reduceByKey** 会按 key 聚合，需要 shuffle，所以在这里切 Stage。**saveAsTextFile** 是 Action，触发执行。PPT 里 WordCount 也强调 **flatMap/map/reduceByKey** 是 Transformation，**saveAsTextFile** 是 Action。

题2：只有 map/filter/count，Stage 怎么切？

题目：HDFS 输入有 3 个 block，程序如下。问几个 Job、几个 Stage、几个 Task？

```
val logs = sc.textFile("hdfs://logs")
val errors = logs
    .map(parseLog)
    .filter(_.level == "ERROR")

errors.count()
```

答案：

```
1 个 Job
1 个 Stage
Stage 1: textFile + map + filter + count
Task 数 = 3
```

详解：`map` 和 `filter` 都是窄依赖，没有 shuffle，所以不会切 Stage。`count()` 是 Action，会触发一个 Job。因为输入有 3 个 HDFS block，所以这个 Stage 有 3 个 Task。

考场写法：没有 **shuffle**，全链路可以放在一个 **Stage** 里流水线执行。

题3：两个 shuffle 怎么切 Stage？

题目：HDFS 输入有 4 个 block，程序如下，shuffle 分区数都设为 3。问 Stage 怎么切？

```
val rdd = sc.textFile("hdfs://input")
    .map(parse)
    .map(x => (x.user, x.value))
    .reduceByKey(_ + _, 3)
    .filter(_._2 > 100)
    .groupByKey(3)

rdd.saveAsTextFile("hdfs://output")
```

答案：

```
1 个 Job
3 个 Stage

Stage 1:
    textFile + map + map + reduceByKey 的 shuffle write
    Task 数 = 4

Stage 2:
    reduceByKey 的 shuffle read + filter + groupByKey 的 shuffle write
    Task 数 = 3
```

```
Stage 3:  
  groupByKey 的 shuffle read + saveAsTextFile  
Task 数 = 3
```

详解：**reduceByKey** 是第一次 shuffle，切一次 Stage。**groupByKey** 是第二次 shuffle，再切一次 Stage。**filter** 是窄依赖，不单独切 Stage。

口诀：几个 **shuffle**，通常就会多切几个 **Stage**。

题4：判断窄依赖还是宽依赖

题目：判断下面操作是窄依赖还是宽依赖，并说明是否会切 Stage。

```
map  
filter  
flatMap  
reduceByKey  
groupByKey  
distinct  
join
```

答案：

```
map: 窄依赖, 不切 Stage  
filter: 窄依赖, 不切 Stage  
flatMap: 窄依赖, 不切 Stage  
  
reduceByKey: 宽依赖, 切 Stage  
groupByKey: 宽依赖, 切 Stage  
distinct: 通常宽依赖, 切 Stage  
join: 通常宽依赖, 切 Stage
```

详解：窄依赖是父 RDD 的每个 partition 最多被一个子 partition 使用；宽依赖是父 partition 可能被多个子 partition 使用，需要 shuffle。PPT 里明确区分了 narrow dependency 和 wide dependency，并把依赖关系作为 Spark 执行方式的关键。

考试最短句：**map/filter** 这种一对一传下去的是窄依赖；**reduceByKey/groupByKey/join** 这种要按 key 重新分组的是宽依赖。

题5：join 到底是窄依赖还是宽依赖？

题目：判断下面两种 join：

情况 A：两个 RDD 没有相同 partitioner。

```
rdd1.join(rdd2)
```

情况 B：两个 RDD 都已经按相同 key 使用相同 partitioner 分区。

```
rdd1.partitionBy(p).join(rdd2.partitionBy(p))
```

问 join 是窄依赖还是宽依赖？

答案：

情况 A：

join 通常是宽依赖，需要 shuffle，会切 Stage。

情况 B：

如果两个 RDD 已经按相同 partitioner 共分区，
同一个 key 已经在对应分区内，
join 可以避免 shuffle，可看成窄依赖/不再新增 shuffle。

详解： join 的核心是让相同 key 的数据碰到一起。默认情况下，不同 RDD 的相同 key 不一定在同一个 partition，所以要 shuffle。但如果两个 RDD 已经按相同 partitioner 分好了区，相同 key 已经对齐，就不需要再打乱数据。

考试写法： join 通常是宽依赖；只有在两个父 RDD 已经按相同 partitioner 共分区时，才可能避免 shuffle。

题6：HBase 输入时第一阶段 Task 数怎么算？

题目： Spark 从 HBase 读取一张表，该表被切成 6 个 Region。程序如下，reduceByKey 设置为 4 个分区。问 Stage 和 Task 数。

```
val rdd = sc.newAPIHadoopRDD(hbaseConfig)
val pairs = rdd.map(row => (extractKey(row), 1))
val result = pairs.reduceByKey(_ + _, 4)
result.saveAsTextFile("hdfs://output")
```

答案：

1 个 Job
2 个 Stage

Stage 1:

HBase scan + map + shuffle write

Task 数 = 6，因为输入来自 6 个 HBase Region

Stage 2:

shuffle read + reduceByKey + save

Task 数 = 4, 因为 reduceByKey 设置了 4 个 shuffle partition

详解： PPT 里讲第一阶段 Task 数通常由 HDFS block 或 HBase region 个数决定。这里输入不是 HDFS，而是 HBase，所以第一阶段看 Region 数。

题7：多个 Action 会触发几个 Job？

题目： HDFS 输入有 5 个 block，程序如下。问会触发几个 Job？如果没有 cache，会发生什么？

```
val lines = sc.textFile("hdfs://logs")
val errors = lines.filter(_.contains("ERROR"))

val n = errors.count()
errors.saveAsTextFile("hdfs://errors")
```

答案：

2 个 Action，所以触发 2 个 Job。

Job 1: errors.count()

Job 2: errors.saveAsTextFile()

如果没有 cache：

errors 这条 RDD lineage 会被执行两次。
第一次为了 count 执行；
第二次为了 saveAsTextFile 再执行。

如果加了：

```
errors.cache()
```

则第一次计算后可以缓存，第二个 Action 复用缓存结果。

详解： Spark 是惰性执行，Transformation 只记录依赖关系，Action 才触发执行。PPT 里明确说 Action 是触发程序分布式执行的算子。

题8：根据代码画 DAG，并标 Stage

题目：给出代码：

```
val a = sc.textFile("hdfs://input")
val b = a.map(parse)
val c = b.filter(_.valid)
val d = c.map(x => (x.city, x.money))
val e = d.reduceByKey(_ + _)
val f = e.filter(_._2 > 1000)
f.collect()
```

要求：写出 DAG，并标出 Stage。

答案：

DAG:

```
textFile
-> map(parse)
-> filter(valid)
-> map(city, money)
-> reduceByKey
-> filter(total > 1000)
-> collect
```

Stage 划分：

Stage 1:

textFile -> map -> filter -> map -> reduceByKey 的 shuffle write

Stage 2:

reduceByKey 的 shuffle read -> filter -> collect

详解：前面的 **map/filter/map** 都是窄依赖，可以和前面的读取放在同一个 Stage。**reduceByKey** 需要按 city 聚合，会发生 shuffle，所以切 Stage。**collect()** 是 Action，触发 Job。

题9：Task 类型怎么判断？

题目：Spark WordCount 中有两个 Stage：

```
Stage 1: textFile + flatMap + map + shuffle write
Stage 2: shuffle read + reduceByKey + saveAsTextFile
```

问这两个 Stage 中的 Task 分别是什么类型？

答案：

```
Stage 1: ShuffleMapTask
Stage 2: ResultTask
```

详解：PPT 里提到 Spark 本质上主要有两种 Task：ShuffleMapTask 和 ResultTask。前面的 Stage 负责产生 shuffle 输出，所以是 ShuffleMapTask；最后一个 Stage 负责得到 Action 的结果或写出结果，所以是 ResultTask。

考试写法：中间 Stage 通常是 ShuffleMapTask，最后一个 Stage 通常是 ResultTask。

题10：Stage 失败后重算哪些数据？

题目：有如下流程：

```
HDFS -> RDD1 -> map -> RDD2 -> filter -> RDD3 -> reduceByKey -> RDD4 ->
save
```

假设 RDD3 的一个 partition 丢失，Spark 如何恢复？如果 RDD4 的某个 shuffle 输出丢失，又如何恢复？

答案：

```
RDD3 的一个 partition 丢失：
    因为 map/filter 是窄依赖，
    Spark 可以根据 lineage 从对应的 HDFS block / 父 partition 重新计算这个
    partition。

RDD4 的某个 shuffle 输出丢失：
    需要重新计算产生该 shuffle 输出的上游 ShuffleMapTask，
    可能涉及重新读取对应父 partition 并重新做 shuffle write。
```

详解：RDD 的核心是 lineage，可以根据计算血统自动重构。窄依赖丢一个 partition，通常只重算相关的一小段；宽依赖涉及 shuffle，恢复成本更高，因为下游可能依赖上游多个 partition 的 shuffle 输出。Spark PPT 里强调 RDD 可溯源、失效后可通过 lineage 自动重构。

最后压成考场版就是：

1. 先找 Action：几个 Action 通常几个 Job。
2. 从后往前找 Shuffle：每遇到 reduceByKey/groupByKey/join/distinct 就切 Stage。
3. Stage 内窄依赖合并：map/filter/flatMap 不切。
4. 第一阶段 Task 数看输入 block/region。
5. Shuffle 后 Stage 的。
4. 第一阶段 Task 数看输入 block/region。

5. Shuffle 后 Stage 的 Task 数看 shuffle partition。
6. join 默认宽依赖；共分区时可能不 shuffle。